Family Office AI Agent: Multi-Agent RAG for Integrated Financial Planning

Zhimei Wang
Columbia University
New York, NY, USA
zw3099@columbia.edu

Abstract

Individuals across the income spectrum face complex, interdependent financial planning decisions spanning tax optimization, investment allocation, and estate planning. Yet access to integrated, professional-level guidance remains expensive and out of reach for many. We present the **Family Office AI Agent**, a multi-agent system that combines Retrieval-Augmented Generation (RAG) with domain-specific reasoning to deliver integrated, evidence-grounded financial planning advice for users at all income levels—from low-income households maximizing tax credits and emergency savings to high-net-worth families managing trust structures and estate exemptions. The system routes user queries to specialized modules—tax, investment, and estate—each backed by a curated FAISS vector store (1,118 documents total). Module outputs are synthesized by an LLM into a unified, cited recommendation. Evaluated on 78 AICPA-style multiple-choice questions and 15 open-ended planning scenarios, the RAG-augmented agent achieves 87.2% accuracy versus an 83.3% direct-LLM baseline, and an average LLM-as-judge score of 7.4/10 on scenario quality.

All code, evaluation scripts, and results are available at:
GitHub: https://github.com/zhimw/midterm-project
Demo Video: https://drive.google.com/file/d/1abxGlKB4Tk8-rM6I8qK5ftHTNwEhkn_s/view

1 Introduction

Financial planning is a universal need, yet access to integrated, professional-level guidance is expensive and largely inaccessible. A low-income family may need help claiming the Earned Income Tax Credit (EITC) and building an emergency fund; a middle-class household may need to decide between a Roth and traditional IRA; a high-net-worth family may need coordinated trust structures and estate tax mitigation. In all cases, optimal decisions span multiple domains—taxes, investments, and estate planning—and the cost of professional advisors means most people navigate these decisions without adequate support.

Large Language Models (LLMs) show promise for democratizing expert reasoning, but suffer from knowledge cutoffs, hallucination, and lack of domain grounding [1]. Retrieval-Augmented Generation (RAG) addresses these shortcomings by conditioning LLM responses on retrieved, curated documents [1]. Multi-agent architectures further improve complex task performance by decomposing queries into specialized sub-tasks [2].

We combine these two paradigms in the **Family Office AI Agent**: a full-stack system that (1) classifies user queries via an LLM router, (2) retrieves relevant knowledge across three domain-specific FAISS indices, (3) runs three specialized reasoning modules in parallel, and (4) synthesizes a cited, integrated recommendation. The system uses Google Gemini 2.5 Flash as its LLM backend.

2 Problem Description

2.1 Target Users and Scenarios

The system is designed to serve US individuals and families across the full income spectrum. Representative planning scenarios include:

Low-income users:

- A 28-year-old single parent earning $32K/year who wants to understand EITC eligibility, the Child Tax Credit, and how to start an emergency fund in a high-yield savings account.
- A 35-year-old gig worker with no employer benefits asking how to reduce self-employment tax, whether a SEP-IRA makes sense, and how to avoid an underpayment penalty.

Middle-income users:

- A 40-year-old teacher with a $95K salary asking whether to contribute to a Roth 403(b) or traditional 403(b), and how to balance student loan repayment with retirement savings.
- A 45-year-old couple with $180K combined income and a 529 plan asking how to optimize college savings alongside their mortgage interest deduction.

High-income / high-net-worth users:

- A 45-year-old tech executive with $1.2M income and a concentrated RSU position seeking to minimize AMT and NIIT exposure while growing wealth.
- A 52-year-old S-Corp business owner planning an exit event and intergenerational wealth transfer before the 2026 estate tax exemption sunset.

EECS6895 '25, Spring 2026, New York, NY
2025.

2.2 Research Questions

1. Can a RAG-augmented multi-agent system outperform a direct LLM call on structured financial knowledge benchmarks?
2. Can the system produce sufficiently comprehensive, grounded responses to open-ended, multi-domain planning scenarios?

3 System Architecture

Figure 1 illustrates the end-to-end pipeline.

```
User Profile (age, income, assets, goals)
        |
   Router Agent
  /      |      \
Tax    Invest  Estate
Module  Module  Module
  \      |      /
   Synthesizer (LLM)
        |
 Response + Citations
```

Figure 1. System pipeline. Modules run in parallel via ThreadPoolExecutor.

The backend is a **FastAPI** application exposing a REST API. The frontend is a **React 18 / TypeScript** single-page application with a 4-step intake form and a chat interface that displays recommendations, structured breakdowns, and evidence citations.

3.1 Session and Memory Management

Each user session is identified by a UUID. An in-memory `ConversationMemory` store maintains: (1) the user profile (all intake form fields), (2) conversation history (role, content, timestamp), and (3) a recommendations history with module attribution. The full conversation history is prepended to each LLM call so that follow-up questions are interpreted in context.

4 Agent Design

4.1 Router

The `AgentRouter` sends the user query to an LLM with a JSON-mode prompt that classifies it into one or more of three modules: *tax optimization*, *investment allocation*, and *estate planning*. Temperature is set to 0 for determinism. A robust fallback parser handles truncated or malformed JSON output, defaulting to all three modules if parsing fails. On average across our test set, 72% of queries invoked more than one module.

4.2 Domain Modules

Each module follows a four-step pattern:

1. **Retrieve** top-k documents from the domain FAISS index via semantic similarity search.
2. **Build context** by formatting retrieved documents as `[doc_id]\ntext` blocks.
3. **LLM reasoning**: a domain-specific system prompt + user profile + retrieved context is sent to the configured LLM provider.
4. **Rule-based extraction**: structured metadata (e.g., risk profile, tax triggers, estate planning flags) is extracted from the LLM response using heuristics.

The three modules differ in their domain knowledge, system prompts, and extraction logic:

Tax Module identifies planning triggers such as NIIT exposure (income >$200K), top-bracket status (income >$578K), and 1031 exchange eligibility based on the user profile. It extracts bullet-point strategies from the LLM response.

Investment Module computes a risk profile (Aggressive Growth / Growth with Tax Optimization / Conservative Income-Focused / Balanced) from the user's risk tolerance and time horizon, recommends asset allocation percentages, and calculates a heuristic tax efficiency score (0–100).

Estate Module computes the total estate value from the assets dict, identifies planning triggers (estate >$13.61M, life events like marriage, birth, or divorce), and recommends trust structures.

4.3 Synthesizer

The `RecommendationSynthesizer` receives all module outputs and calls the LLM with a meta-prompt that instructs it to produce a unified, integrated recommendation. It handles two response modes:

- **Multiple-choice mode**: triggered when a query starts with the standardized test prefix; forces the LLM to emit a single letter (A/B/C/D).
- **Conversational mode**: produces a full narrative response incorporating all module outputs, appending a mandatory risk disclaimer.

The synthesizer also extracts structured breakdown data (tax strategies list, investment allocation dict, estate triggers list) for the frontend visualization panels.

5 Tools, RAG, and Components

5.1 Knowledge Base Construction

We curated three domain-specific knowledge bases:

Tax knowledge was extracted from IRS PDFs using `pdfplumber`. Investment knowledge was parsed from publicly available capital market assumption reports. Estate knowledge was collected via a web scraper targeting authoritative estate planning resources. All documents are stored as JSONL with fields `doc_id`, `category`, and `text`.

5.2 RAG System

The RAG layer consists of:

Table 1. Knowledge base statistics.

Domain	Docs	Sources
Tax	838	IRS Pub. 17, 523, 526, 550, 590a, 590b
Investment	168	Fidelity & Vanguard capital market reports
Estate	112	Curated planning references
Total	**1,118**	

- **Embedding model**: `all-MiniLM-L6-v2` from Sentence Transformers (local, no API cost).
- **Vector store**: FAISS via `langchain_community`, one index per domain.
- **Retrieval**: top-$k = 5$ semantically similar documents per query.
- **Evidence formatting**: retrieved documents are returned to the frontend as structured evidence objects `{doc_id, category, snippet, full_text}` for citation display.

FAISS indices are persisted to disk after first build and loaded from cache on subsequent starts, reducing startup time from ~60s to ~3s.

5.3 LLM Client

The system uses **Google Gemini 2.5 Flash** as its LLM backend. JSON mode is used for the router and MC synthesizer to guarantee parseable output. The `disable_thinking` flag suppresses Gemini's internal chain-of-thought for short, deterministic tasks (routing, multiple-choice answers) to prevent output truncation.

6 Experiments and Example Use Cases

6.1 Evaluation Framework

We designed a two-track evaluation:

Track 1 — AICPA Multiple-Choice Benchmark. 78 AICPA-style financial planning questions with ground-truth answers (A/B/C/D). Questions were submitted to both the RAG+Agent pipeline and a direct LLM call (no retrieval) that relies only on the model's parametric knowledge.

Track 2 — Open-Ended Scenario Evaluation. 15 rich planning scenarios, each with a full user profile, a complex open-ended question, and a grading rubric. Evaluated by two metrics: (a) *Keyword coverage*: fraction of expected domain concepts found in the response. (b) *LLM-as-judge*: a separate LLM call rates the response 1–10 against the rubric, returning a score and one-sentence justification.

Table 2. Track 1 results: AICPA multiple-choice accuracy (78 questions).

System	Correct	Accuracy
RAG + Agent	68/78	**87.2%**
Direct LLM (baseline)	65/78	83.3%
Improvement	+3	+3.9 pp

Table 3. Track 2 results: open-ended scenario quality (15 scenarios).

Metric	Score
Avg. keyword coverage	67.7%
Avg. LLM-as-judge score	7.4 / 10

6.2 Results

RAG grounding provides a consistent accuracy lift (+3.9 percentage points) on factual financial knowledge questions. Scenario responses score 7.4/10 on average, indicating strong domain coverage while highlighting occasional gaps in specific sub-topics (e.g., deferred compensation, Texas state tax nuances).

6.3 Example Interaction

A tech executive (age 45, $1.2M income, $5M assets, California) asks: *"How should I handle my RSU vesting to minimize taxes?"*

The router selects the *tax optimization* and *investment allocation* modules. The tax module retrieves documents on AMT, NIIT, and California state tax, producing strategies including systematic RSU selling, direct indexing for tax-loss harvesting, and charitable donation of appreciated shares. The investment module recommends shifting 15% of the portfolio to California municipal bonds to eliminate NIIT on fixed income. The synthesizer integrates both outputs into a narrative that also flags the 2026 estate tax exemption sunset, triggering the estate module's recommendation to explore a Dynasty Trust. The frontend displays this alongside six source citations and a real-time investment allocation chart.

7 Limitations and Future Improvements

7.1 Current Limitations

Knowledge currency. The corpus reflects 2024 tax-year data. Annual IRS publication updates (e.g., inflation-adjusted brackets, contribution limits) require re-ingestion and re-indexing.

No persistent storage. Session data is held in-memory and lost on server restart. A production deployment would require a database backend for session persistence.

No authentication or authorization. The API has no access control, making it unsuitable for production deployment without significant additional engineering.

Keyword coverage gaps. Track 2 results show an average keyword coverage of only 67.7%, indicating the system sometimes misses specific planning concepts (e.g., Texas no-income-tax advantage, step-up in basis details).

Scope boundaries. The system covers federal financial planning broadly, but state-specific tax nuances beyond a few prominently documented states (California, New York, Texas) are underrepresented in the corpus.

7.2 Future Improvements

Agentic web search. Adding a real-time search tool (e.g., via the OpenAI Responses API with built-in web search) would allow the agent to fetch current IRS guidance, court rulings, or state tax bulletins on demand.

Persistent memory and user profiles. Moving session storage to PostgreSQL or Redis would enable long-term tracking of a family's financial trajectory, enabling proactive planning alerts (e.g., "your estate value has crossed the exemption threshold").

Expanded corpus. Adding state-specific tax guides for all 50 states and integrating Social Security Administration publications would broaden coverage for retiree scenarios.

Fine-tuning. Instruction-tuning an open-source model (e.g., Llama-3.3) on curated (question, answer) pairs from AICPA practice exams and CPA study materials could improve factual accuracy while reducing API costs.

Multi-turn planning sessions. Extending the synthesizer to maintain a long-horizon planning state (e.g., a structured financial plan that evolves across multiple sessions) would move the system from a Q&A tool toward a true planning agent.

Quantitative scenario modeling. Integrating a tax computation engine (e.g., calling TaxBrain or a local tax library) would allow the system to produce concrete numerical projections (e.g., "tax-loss harvesting saves approximately $24,000 in federal taxes this year") rather than qualitative guidance.

8 Conclusion

We presented the Family Office AI Agent, a multi-agent RAG system for integrated financial planning. By decomposing complex queries across specialized tax, investment, and estate modules—each grounded in curated domain knowledge via FAISS semantic search—the system achieves 87.2% accuracy on AICPA-style questions (vs. 83.3% for a direct LLM baseline) and produces scenario responses rated 7.4/10 by an LLM judge. The modular design, multi-provider LLM support, and full-stack implementation provide a strong foundation for future extensions toward production-grade family office intelligence.

Acknowledgments

This project was completed as part of EECS6895 (Big Data Analytics) at Columbia University, Spring 2026.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[2] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A Survey on Large Language Model based Autonomous Agents. In *Frontiers of Computer Science*, Vol. 18.